DuocUC® | ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Evaluación Parcial 2 — Ejecución Práctica

Instrucciones y Pauta de Evaluación · Caso AutoMarket

FORMA A · SECCIÓN 002D

Sigla	Nombre Asignatura	Tiempo Asignado	% Ponderación	Experiencia de Aprendizaje
DSY1102	Programación Orientada a Objetos	3 bloques	35%	EA2 — Interfaz Gráfica JavaFX, MVC y JSON

---

1. Contexto General y Proyecto Base Entregado

La concesionaria **AutoMarket** requiere una aplicación de escritorio en JavaFX bajo arquitectura **MVC** para visualizar su inventario de vehículos (**Automóviles** y **Motocicletas**) y registrar nuevas unidades en tiempo real.

Se entrega un proyecto Maven base (carpeta `automarket`) que ya compila e inicia una ventana básica. El proyecto entregado ya incluye:

Componente Entregado	Detalle Técnico
`pom.xml` y Arranque	Dependencias configuradas (`javafx-controls` y `gson`) y clases `Main`, `AutoMarketApp` y `HomeView` operativas.
`resources/vehiculos.json`	Arreglo JSON con 10 vehículos iniciales diferenciados por el atributo `"tipo"` (`"automovil"` o `"motocicleta"`).
`model/Vehiculo.java`	Clase abstracta con atributos `id` (int), `marca` (String), `modelo` (String), `precio` (int), `disponible` (boolean), getters/setters y métodos abstractos `getTipo()` y `getEspecificacion()`.
`model/Automovil.java`	Subclase con atributos propios `puertas` (int) y `transmision` (String). Implementa `getTipo()` y `getEspecificacion()`.
`model/Motocicleta.java`	Subclase con atributos propios `cilindradaCc` (int) y `velocidadMaxKmh` (int). Implementa `getTipo()` y `getEspecificacion()`.
Clases a completar	`model/VehiculoManager.java`, `controller/VehiculoController.java` y `view/CatalogoView.java`.

2. Vista Objetivo a Construir

Debes completar `CatalogoView.java` para que la ventana principal presente la tabla de inventario y el formulario de inserción siguiendo esta estructura visual:

AutoMarket — Inventario de Vehículos — □ ×

AutoMarket — Catálogo de Automóviles y Motocicletas

ID	Tipo	Marca	Modelo	Precio	Especificación	Disponible
1	Automóvil	Toyota	Corolla Hybrid	18990000	4 puertas · Automática	true
2	Motocicleta	Yamaha	MT-07	8490000	689 cc · 210 km/h	true
3	Automóvil	Mazda	Mazda 3 Sport	17490000	5 puertas · Mecánica	true

Registrar nuevo vehículo en el catálogo:

Tipo: [Automóvil / Motocicleta ▼] Marca: [_____] Modelo: [_____]

Precio: [_____]

Atributo 1 (Puertas / Cilindrada cc): [____] Atributo 2 (Transmisión / Vel. Máx): [_____]

[Agregar Vehículo]

3. Qué se Solicita Evaluar

1. Lectura y Transformación JSON — 20%

Leer el archivo `vehiculos.json` en `VehiculoManager`, transformar cada registro en una instancia de `Automovil` o `Motocicleta` según corresponda y almacenarlos en una colección polimórfica en memoria.

2. Arquitectura MVC — 15%

Mantener la separación de responsabilidades: la vista solo interactúa con `VehiculoController`, y el controlador delega la gestión de datos en `VehiculoManager`.

3. Visualización en TableView — 20%

Construir el componente `TableView<Vehiculo>` con sus columnas enlazadas al modelo para mostrar todos los vehículos cargados al iniciar la aplicación.

4. Inserción desde Formulario — 20%

Construir el formulario que permita ingresar un nuevo vehículo, validar que los datos no rompan la ejecución, agregarlo a la colección y reflejarlo de inmediato en el `TableView`.

5. Eliminación de Registros (Parte 2) — 15%

Habilitar la remoción del vehículo seleccionado en la tabla mediante un botón de eliminación, delegando la operación por MVC y actualizando la vista en tiempo real sin excepciones.

6. Validación y Excepciones — 10%

Controlar campos vacíos y capturar errores de formato numérico (`try-catch`) para asegurar la estabilidad total del sistema durante toda la interacción.

Parte 2 disponible: Para consultar el requerimiento detallado de eliminación y su diseño de interfaz, revisa el **Anexo: Eliminación de Registros (HTML)** o descarga el **Anexo en PDF**.

4. Obtención del Código Base y Forma de Entrega

```
# 1. Clonar la rama base y entrar al repositorio
git clone -b evaluacion2/base https://github.com/hachimaki-
dev/002D_desarrollo_orientado_a_objeto_2026.git
cd 002D_desarrollo_orientado_a_objeto_2026

# 2. Crear tu rama personal e iniciar el entorno
git checkout -b evaluacion2/Nombre_Apellido
.\setup-javafx.ps1

# 3. Al finalizar: guardar commit, subir tu rama y adjuntar ZIP del proyecto en AVA
git add .
git commit -m "Entrega EP2 - Nombre Apellido"
git push origin evaluacion2/Nombre_Apellido
```

5. Pauta de Evaluación (Rúbrica)

Indicador de Evaluación	Muy buen desempeño (100%)	Buen desempeño (80%)	Desempeño aceptable (60%)	Desempeño incipiente (30%)	No logrado (0%)	Pond.
IE 1. Lectura de JSON y transformación polimórfica Deserializa el archivo JSON e instancia las subclases en una colección polimórfica.	Lee `vehiculos.json` con Gson, discrimina ambos subtipos (`Automovil` y `Motocicleta`) e inserta la totalidad de los registros en una colección polimórfica `Vehiculo` .	Lee el JSON y carga ambos subtipos en la colección polimórfica, con un detalle menor en el manejo del flujo o cierre de recursos que no impide la carga.	Carga únicamente uno de los dos subtipos desde el JSON, o bien omite atributos propios de las subclases al transformar los datos.	Intenta leer el archivo JSON pero la deserialización falla o la colección queda vacía al iniciar.	No implementa la lectura del JSON o escribe los datos de forma estática en el código.	**20%**
IE 2. Aplicación del patrón MVC Separa responsabilidades entre Vista, Controlador y Manager.	La vista se comunica exclusivamente con `VehiculoController` y este delega el acceso a datos y operaciones sobre la colección en `VehiculoManager` .	Respeta la comunicación Vista → Controlador → Manager, con un detalle menor de acoplamiento que no rompe la separación de paquetes.	El controlador existe, pero la vista accede directamente a `VehiculoManager` en alguna de las operaciones (carga o inserción).	Concentra la lectura del JSON o la colección directamente dentro del controlador o de la vista, omitiendo el rol del Manager.	No utiliza el controlador ni el manager; toda la lógica está escrita dentro de la vista.	**15%**
IE 3. Construcción del TableView Configura la tabla y enlaza las columnas con el modelo.	El `TableView<Vehiculo>` presenta las columnas solicitadas enlazadas a los atributos y métodos polimórficos del modelo, mostrando todos los registros al iniciar.	La tabla muestra los registros de ambos subtipos correctamente, pero falta una columna solicitada o un encabezado no es descriptivo.	La tabla muestra los registros, pero dos o más columnas aparecen en blanco por errores en los nombres de las propiedades enlazadas.	El componente `TableView` se declara y agrega a la vista, pero no muestra filas al iniciar la aplicación.	No incorpora un componente `TableView` en la vista.	**20%**
IE 4. Formulario e inserción de nuevos datos Registra nuevas instancias y actualiza el TableView en tiempo real.	El formulario permite crear tanto un `Automovil` como una `Motocicleta` , lo agrega a la colección mediante el controlador y el nuevo registro aparece de inmediato en el `TableView` .	El nuevo vehículo se agrega y aparece en el `TableView` para ambos subtipos, pero el formulario no limpia sus campos tras el ingreso o requiere clic adicional.	El formulario solo permite insertar uno de los dos subtipos, o bien inserta el objeto en la colección sin refrescar visualmente el `TableView` .	Los controles del formulario están dibujados en la vista, pero el evento del botón no logra crear ni insertar el objeto.	No construye el formulario de ingreso ni el evento de inserción.	**20%**
IE 5. Eliminación de registros y actualización (Parte 2 — Anexo) Elimina el registro seleccionado del TableView y de la colección en memoria.	Permite seleccionar un vehículo en el `TableView` , delega la eliminación a través de `VehiculoController` hacia `VehiculoManager` , remueve el registro de memoria, refresca la tabla al instante y maneja de forma segura pulsaciones sin selección activa.	Elimina el vehículo seleccionado respetando el flujo MVC y actualiza la tabla, pero no previene el caso de pulsar el botón sin una fila seleccionada (provocando alertas residuales en consola).	Remueve la fila visualmente de la tabla pero no de la colección en el Manager, o la vista realiza la remoción saltándose el Controlador.	Dibuja el control de eliminación en la vista, pero su evento no logra obtener el elemento seleccionado o lanza `NullPointerException` que detiene la interacción.	No implementa la funcionalidad de eliminación de registros.	**15%**
IE 6. Validación, excepciones e integración Controla errores de ingreso y ejecuta el flujo sin caídas.	El proyecto compila e inicia sin errores; valida campos vacíos y captura errores de conversión numérica (`try-catch`) sin que la aplicación se detenga ante entradas inválidas.	La aplicación ejecuta el flujo completo y controla conversiones numéricas, pero omite validar algún campo de texto vacío antes de registrar.	La aplicación compila y ejecuta el flujo normal, pero lanza una excepción no controlada en consola si se ingresa texto en un campo numérico.	La aplicación inicia con fallos en más de una operación principal o se bloquea durante la interacción.	El proyecto no compila o se detiene al intentar arrancar la ventana.	**10%**
					Total Evaluación Parcial 2	**100%**